



THU
Technische
Hochschule
Ulm

HENSOLDT

Bachelor thesis at Ulm University of Applied Sciences

Department of Computer Science

Degree Program Informatik

Presented by Gabriel Zimmermann

April 2026

1. Reviewer: Prof. Dr.-Ing. Klaus Baer
2. Reviewer: Prof. Dr.-Ing. Philipp Graf

Place, Date, Signature

Diese Abschlussarbeit wurde von mir selbständig verfasst. Es wurden nur die angegebenen Quellen und Hilfsmittel verwendet. Alle wörtlichen und sinngemäßen Zitate sind in dieser Arbeit als solche kenntlich gemacht.

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references. Place, date and signature

Ort, Datum und Unterschrift

Abstract

Something something what approaches are there to setup plugin systems in Rust. What are the pros and cons...

Contents

Abstract	3
I) Metadata University.....	11
II) Introduction	11
III) Overview	11
III.A) Goal	11
III.A.a) Static	11
III.A.b) Dynamic	11
IV) Methods	12
IV.A) Performance	12
IV.A.a) Tools	12
IV.A.b) Benchmarks	12
IV.A.c) Evaluation	12
IV.B) Development complexities	12
IV.C) Limitations	12
IV.D) Safety	12
IV.E) interoperability	12
V) Notes that i learned about dynamic plugin approach =w=	13
V.A) Rust ABI	13
VI) Safety (Notes)	14
VI.A) Levels	14
VI.A.a) DO-178C	14
VII) Performance (Notes)	14
VII.-a) Sources	15
References	15

Plugin System's in Rust

Gabriel Zimmermann

I. Metadata University...

```
fn main() {  
    println!("Hello world!")  
}
```

Listing 1. Rust Code

II. Introduction

Analysis of dynamic plug-in systems in rust. Comparison of different approaches focusing the pros and cons.

Explain ABI and why it's important.

Calling Conventions, Where is Data saved and all that...

Some Source for this: <https://doc.rust-lang.org/reference/items/static-items.html#static-items> ??

Rust ABI is not stable, which means that we cannot guarantee that the same binary will work with different versions of the compiler. This is a problem for dynamic plugins, as they need to be compiled separately from the main application.

<https://doc.rust-lang.org/reference/abi.html>

Add some more details about "other" ABIs (cough C??) and state that that has been the common way of building systems and apps in the past!

III. Overview

Problem: Rust has unstable ABI, which makes the same approach of C not functional.

Therefore, this paper looks at various approaches.

This can be separated into static and dynamic approaches. Static requires the recompilation of binaries.

A. Goal

Analysis of these things

a) *Static*:

I can only mention here, that we can enable/disable parts of code during compile time, yes? aka `#[cfg(feature = "enable_insane_feature")]` or `cfg!(feature = "enable_insane_feature")` -> extra careful during development, that all required features are structured without accidental deactivations??

b) *Dynamic*:

Loading .so w/ libloading -> can be used for c and rust (but both un- and stable rust ABI should work here)

- unstable Rust ABI
- "stable" Rust ABI Crate
- C ABI
- rust bridge

I should probably explain `#[no_mangle]` ...

Due to dyn compability, we must use `&self` or `&mut self`

Mention speed up compile times?

IV. Methods

Goal of the research was to analyse the following points:

- performance
- development complexities
- limitations
- safety
- interoperability

A. Performance

Only “pure” quantitative measurement will be the performance and maybe safety (using safety levels).

a) Tools:

Using criterion and gungrau benchmarks. Not a perfect measurement, but it should give us enough hints about what happens behind the scenes.

Using black_box, we can ensure that the internal code isn’t hyper-optimized by the compiler, which can lead to more accurate benchmarks.

Gungrau uses valgrind internally.

b) Benchmarks:

To test all of this, we use a simple temperature simulation, where a random amount of sensors will pick up the data.

Benchmarks are run with the following set of configurations:

- run app “regular” with black_box
- run average temperature fusion code with black_box once (sensors: 1 vs. 1000000)
- run average temperature fusion code without black_box (sensors: 1 vs. 1000000)

c) Evaluation:

Gungrau helps us see the internal required instructions on “my” CPU. This result should correlate with the actual run time using criterion. We can then reason about the expected overhead of a chosen approach.

B. Development complexities

Subjective, but also Qualitative Measurement

How will this impact a development team? This might just be an extra Evaluation of Limitations

C. Limitations

Might be part of “complexities”??

Qualitative Measurement

D. Safety

Attempting Quantitative by using safety levels.

Rust Language generally very safe. Errors often need to be “forced” or careless usage of unwinding. [1]

E. interoperability

How flexible are the presented approaches? Can they be linked with other libraries?

V. Notes that i learned about dynamic plugin approach =w=

A. Rust ABI

Same Compiler Version! Version 2024 requires some “unsafe” code...

Loading requires some unloading.

After loading, we need to “pin” the data, otherwise it will be dropped after the function call, which can lead to a segfault. [2]

Check out ‘dlopen’ crate? <https://docs.rs/dlopen/latest/dlopen/>

VI. Safety (Notes)

A. Levels

- SIL 1-4 (Safety Integrity Level)
- ASIL A-D (Automotive Safety Integrity Level)
- DAL A-E (Design Assurance Level)
- ...

a) DO-178C:

Software Considerations in Airborne Systems and Equipment Certification

- Compiler by Ferrocene is currently the only qualified one
- Whole toolchain needs to be qualified, not just the compiler (static analyzers, coverage tools, linkers), otherwise extensive “tool validation” is required
- ...

Problem for this thesis: I am using crates that are not qualified. For now, we can summarize all rust crates as “not qualified”...

My approach would be: Just mention this, and then say that we will not focus on this aspect in this thesis, but it is an important topic for future work.

Some sources for this topic:

- <https://ferrocene.dev/> (ISO 26262 (ASIL D), IEC 61508 (SIL 4) and IEC 62304 available targeting Linux, QNX Neutrino or your choice of RTOS)
- <https://ferrous-systems.com/blog/ferrocene-libcore-news-release/>
- <https://rustfoundation.org/security-initiative/>
- <https://rustfoundation.org/media/announcing-the-safety-critical-rust-consortium/>

VII. Performance (Notes)

Quantative measurements: Use Benchmarks to evaluate Overhead for function calls using crates “Gungraun” (internally callgrind/valgrind) and “Criterion” (Execution Time).

This approach is not perfect, but it should give us enough hints about what happens behind the scenes. Benchmarks will be run with and without “black_box”, which ensures that the internal code isn’t hyper-optimized by the compiler. “black_box”: Compiler treats the output as “unknownable” (hint: arguments are unpredictable, results are volatile).

Finally, the results of the benchmarks do need to be reasoned about, as the actual performance can be influenced by various factors, such as the specific hardware architecture, the complexity of the code being executed, and the specific implementation of the dynamic loading mechanism.

Using ‘RUSTFLAGS=“-C target-cpu=native” cargo bench’ to ensure the best results for my system. When using dynamic loading, we ensure that the code is built in “release” mode. Benchmarks are separated into “Overhead” and “Actual Function Call”. For example, the libloading overhead for (unstable) RUST ABI is 230µs, but the actual function call performs at similar speeds compared to static implementations.

NOTE: “native” is not supported for all CPUs. Maybe choose a specific architecture? Current: AMD Ryzen™ 9 3900X x 24

TODO: Refill data with more fine-grained results... Also added Dynamic!!
(‘Box::new(AverageFuser{ })’)

TABLE I
Execution times

	Static	Box<dyn Fuser>	Unstable Rust ABI
Creation	239.33 ps	241.20 ps	210.68 μ s
Creation (blackbox)	244.64 ps	248.88 ps	214.22 μ s
Function (single)	1.2431 ns	1.2153 ns	14.949 ns
Function (single, blackbox)	2.5452 ns	2.4973 ns	15.205 ns
Function (multiple)	870.21 μ s	848.04 μ s	
Function (multiple, blackbox)	908.36 μ s	885.02 μ s	

TABLE II
Estimated Cycles

	Static	Unstable Rust ABI
Creation	-	6849
Creation (blackbox)	-	6826
Function (single)	128	7080
Function (single, blackbox)	293	7097
Function (multiple)	259	15012499
Function (multiple, blackbox)	16500892	15012550

TABLE III
Instruction Count

	Static	Unstable Rust ABI
Creation		
Creation (blackbox)		
Function (single)		
Function (single, blackbox)		
Function (multiple)		
Function (multiple, blackbox)		

a) Sources:

- “Scientific Methodology and Performance Evaluation for Computer Scientists” <https://github.com/alegrand/SMPE>
- “Code Performance” <https://www.sciencedirect.com/topics/computer-science/code-performance>
- “Profiling Tools” <https://www.sciencedirect.com/topics/computer-science/profiling-tool>
- “black_box” (Rust Docu): https://doc.rust-lang.org/beta/std/hint/fn.black_box.html

References

- [1] T. R. P. Developers, “The Rustonomicon: Unwinding.” Accessed: Feb. 24, 2026. [Online]. Available: <https://doc.rust-lang.org/nomicon/unwinding.html#unwinding>
- [2] Nykau, “Why does the object instance I retrieve via libloading make the program to segfault despite lifetime constraints?.” Accessed: Feb. 27, 2026. [Online]. Available: <https://stackoverflow.com/questions/74907868/why-does-the-object-instance-i-retrieve-via-libloading-make-the-program-to-segfa>